

OPERATOR

① What is an Operator?

Math
+, -, *, / (modulus), <, >, =
symbols

- An operator is a special character or symbol that used to perform mathematical or logical operations based on the operand.

expression
int y / 2 - (5 + 4)

② What is an Operand?

- An operand can be a constant or a variable *fixed*
- It is a value on which any operator works

operand
a, b, 2
operator
+

2 + 5 - 4
operand

Example

assignment
sum = a + b + 2 *RHS*

Operand → sum, a, b, 2

Operator → =, +

③ What is an Expression?

- Expression is a combination of operator and operands, and performs a specific action which gives a single value after processing *later*

Later
 $(a+b)^2 = a^2 + 2ab + b^2$

Later
 $a=2$
 $b=4$
 $\frac{+ b \sqrt{b-4a}}{2a}$

Type of Operators (based on number of operands)

1. Unary Operator

2. Binary Operator

3. Ternary Operator

advantages
simple and easy to use

✓ Unary Operator

operates on a single operand

Example:

$a++$

increment

$a--$

decrement

$a = -b$

compound
two work

$a = -(a + b)$

1 operand

✓ Binary Operator

operates on two operands

Example:

$a = a + b$

$a = a / b$

$a > b$

$a == b$

4 + 5

Ternary Operator

operates on three operands, also known as conditional operator, (?:)

Example:

expression1 ? expression 2: expression 3

expression1 = condition

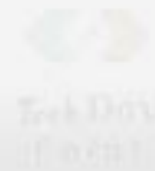
if expression1 is true then execute expression 2

if expression1 is false then execute expression 3

```
int a=2, b=3;  
x = (a > b) ? a : b
```

Type of Operators (based on operations)

- ✓ 1. Arithmetic Operators
- ✓ 2. Increment/Decrement Operators (++ and --)
- ✓ 3. Relational Operators (< , > , =)
- ✓ 4. Logical Operators (and , or , not)
- ✓ 5. Bitwise Operators & , ^ , ~
- ✓ 6. Assignment Operators
- ✓ 7. Special operators (& , >)



Q! = if a and b are same then false int 2005: n [5]

Name of Operator	Arithmetic Operators
Arithmetic Operators	+ , - , * , / , % Remainder returns Quotient
Increment/Decrement Operators	++ , -- (Label)
Relational Operators	> , < , >= , <= , == , != not equal to $a \neq b = \text{False}$

output = true or false $\rightarrow a \geq b$
 $a == b$
 $\neg$

$$J = 1$$
$$F = 0$$

$a < b$
 $a == b$
assignment
equality operator

Logical Operators

&&, ||, !

Bitwise Operators

&, ^, |, ~, >>, <<

Assignment Operators

=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=

Special operators

?: & * sizeof() ,

Type of Operators in C

Based on Number of Operands

1. Unary Operator
2. Binary Operator
3. Ternary Operator

Based on Operations

1. Arithmetic Operators
2. Increment/Decrement Operators
3. Relational Operators
4. Logical Operators
5. Bitwise Operators
6. Assignment Operators
7. Special operators



PRECEDENCE & ASSOCIATIVITY

$$9 + 5 \times 4$$

Solution 1:

$$(\underline{9 + 5}) \times 4$$

$$14 \times 4 = 56$$

Solution 2:

$$9 + (\underline{5 \times 4})$$

$$9 + 20 = 29$$



Precedence

- **Precedence** determines, which operator will be **evaluated first**, in case when there are **more than one operators** used in the expression.

9 + 5 * 4

29



Precedence	Operator
Highest	*, / , %
Lowest	+, -

$10 / 5 * 4$

Associativity

Solution 1: Left to Right

$$10 / 5 * 4$$

$$2 * 4 = 8$$

Solution 2: Right to Left

$$10 / 5 * 4$$

$$10 / 20 = 0.5$$



Associativity

- If the **precedence operators are same**, Associativity determines whether the operator evaluation will begin from **left to right** or **right to left**

10 / 5 * 4

Precedence	Operator	Associativity
Highest	*, / , %	Left to right
Lowest	+, -	Left to right

Table of Operator

Precedence and Associativity in C

Precedence of operators decreases from top to bottom.

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
	Logical OR	left to right
? :	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	comma operator	left to right

INCREMENT & DECREMENT OPERATOR

Increment/Decrement Operators

- **Increment operator** increases the value of a integer variable by one
- **Decrement operator** decreases the value of a integer variable by one
- both are unary operator, i.e. need only one operand

Operator	Description
++	Increment Operator
--	Decrement Operator

Example:

Increment

```
int x = 4;  
x++;
```

x++ is same as $x = x + 1$

Decrement

```
int x = 4;  
x--;
```

x-- is same as $x = x - 1$



Tut Data
Point

Type of Increment / Decrement Operator

- **Pre**-Increment Operator

++X;

- **Post**-Increment Operator

X++;

- **Pre**-Decrement Operator

--X;

- **Post**-Decrement Operator

X--;



Pre-Increment/Decrement Operator

- First increment or decrement the value by one then assign it to another variable

Example:

```
int x, y, a = 4, b = 10;
```

```
x = ++a;
```

```
y = --b;
```



Post-Increment/Decrement Operator

- First assign it to another variable then increment or decrement the value by one

Example:

```
int x, y, a = 4, b = 10;
```

```
x = a++;
```

```
y = b--;
```

4

a

10

b

x

y



```
int x, y, p, q, a = 4, b = 10;
```

```
x = ++a;
```

```
y = b--;
```

```
p = a++;
```

```
q = ++b;
```

4

a

10

b

x

y

p

q

Increment/Decrement Operator inside `printf()`

- The order of evaluation of function parameters is **not guaranteed** in C.
- It might be **left to right**, or it might be **right to left**. It's up to the compiler implementation.
- Usually, most of the compilers evaluate the parameters of **`printf()` function** from **right to left**.

```
printf("%d %d %d %d", a++, a-- , ++a, a);
```

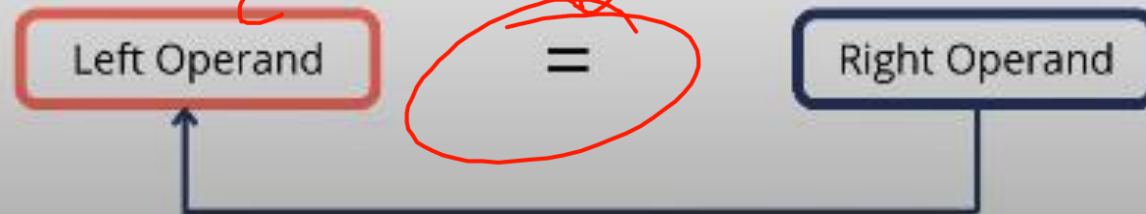




ASSIGNMENT OPERATOR

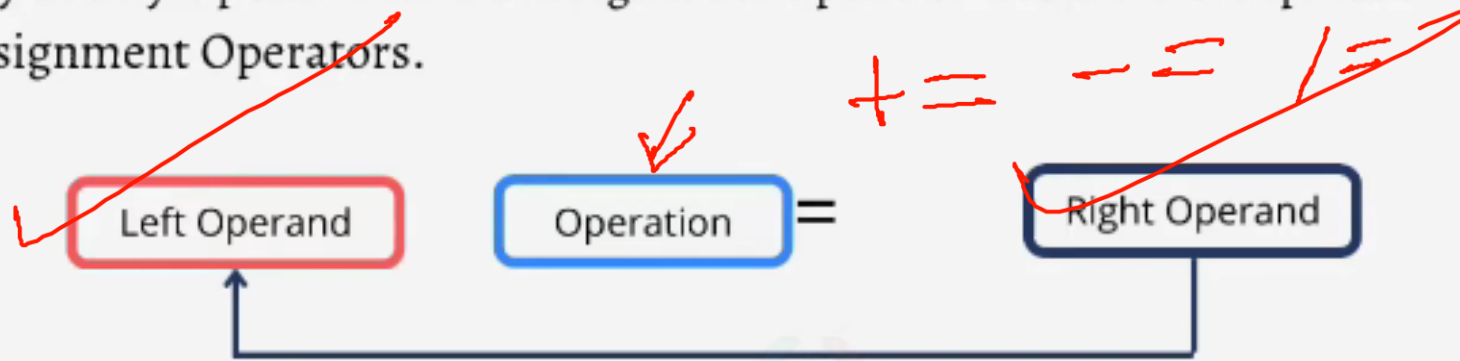
Assignment Operator

- Assignment Operators are used to **assigning** the **value** or **result of an expression** to a **variable**
- Left operand must be a variable and Right operand must be a constant, variable or expression
- Assignment Operators are **Binary operator**, i.e. need two operand
- They have the **lowest precedence** level among all the operators except comma (,)
- Associativity from **right to left**



Compound Assignment Operator

- Any Binary Operator with a Assignment Operator will form Compound Assignment Operators.



Example:

`a = a + 10;`

Simple Assignment Operator

`a += 10;`

Compound Assignment Operator

Handwritten notes in red ink:

- $a + 10$
- $a = a + 10$
- $b + = C$
- $b = b + C$

Assignment Operator

Operator	Description
=	assignment operator
+=	Increment then assign
-=	Decrement then assign
/=	Divide then assign
*=	Multiply then assign
%=	Modulus then assign

$u = y$
 $u = u + y$

$a \% = b$
 $\rightarrow a = a \% b$

~~$b = b \% a$~~

RELATIONAL OPERATOR

Relational Operators

- Relational Operators are used to **comparing two operands** like two integers, characters etc.
- Relational Operators determine the **relationship between operands** and always give the **result in 1 or 0**, where 1 means that the comparison is True and 0 means that the comparison is False
- They are **Binary operator**, i.e. need two operand

Left Operand

Relational Operator

Right Operand

Relational Operators

Operator	Description
=	Is equal to
!=	Is not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to



LOGICAL OPERATOR



Logical Operators

- There are three logical operators in C. - AND(&&), OR(||) and NOT(!)
- AND and OR Logical operators are used to combine two or more conditions.
- NOT operator is used to inverse the condition
- It always give a single result in form of true (1) or false (0)
- && and OR operators are Binary operator, i.e. need two operand
- ! operator is Unary Operator, i.e. need one operand

Left Operand

&&, ||

Right Operand

Combine two or more condition

!

Operand

Inverse the condition

DL = logical gate } AND OR NOT

Logical AND(&&) Operator

- && Operator returns TRUE (1) when all conditions are true and returns FALSE (0) when any one condition is false

Truth Table

A	B	A && B
F 0	0 F ✓	0 F
T 1	0 F ✓	0 F
F 0	1 T ✓	0 F
T 1	1 T	1 T

int a = 5, b = 2;
 $(a < b) \&\& (a == b)$
operand 1 logical AND operand 2

= 0 && 0.
= 0 (False)

Logical OR(||) Operator

- || Operator returns TRUE (1) when one or more conditions are true and returns FALSE (0) when all one conditions are false

$a = 5$
 $b = 5$

A	B	A B
0	0	0 ✓
1	0	1 ✓
0	1	1 ✓
1	1	1 ✓

$a < b$ || $a == b$
 F || F
 $= F$
 $5 < 5$ || $5 == 5$
 F || T
 $= T$

Logical NOT(!) Operator

- ! operator is used to inverse the condition



$$a = 5$$
$$b = 2$$

$$!(a < b)$$
$$!(5 < 2)$$
$$!F = T$$

A	!A
→ 0	1
← 1	0

True

Precedence and Associativity of Logical Operators

Precedence	Operator	Associativity
Highest	!	Right to Left
	&&	Left to right
		Left to right
Lowest		

Assignment

What is the output of the following 'C' program?

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int a = 10, b = 20, c = 5, d = 2, result;

    result = ((a < b) && (a == b)) || (b >= d) && (c <= a) && !(a < c);
    printf("Result is: %d \n", result);

    getch();
    return 0;
}
```

BITWISE OPERATOR

(V.V.P)

Shakti

S. 44 S. 2

Bitwise Operators

byte level
→ 4 = 4 bytes
Binary = 0100
bits
32 bits

- Bitwise operators are used to perform operation or manipulating data at the bit level.

- There are 6 bitwise operators in C -

Bit is the smallest unit to store data in computer memory,
Computer stores information in the form of bytes (1s and 0s) in all its memory.

- Bitwise AND (&)

- Bitwise OR (|),

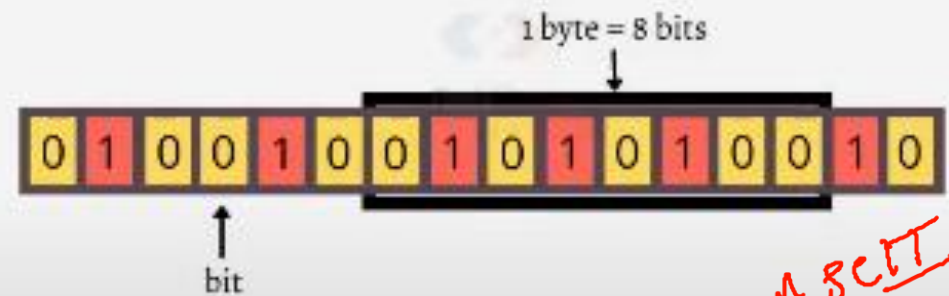
- Bitwise XOR (^),

- Bitwise NOT (~),

- Left Shift Operator (<<)

- Right Shift Operator (>>)

- We can only apply these operators on only Integer and Character values but can't apply on float or double values



1 byte = 8 bits

4 = 4 × 8 = 32

A = 65 = 65₁₀ = 01000001₂

Bitwise Operators

- The operation with Bitwise operators is faster and require less memory
- Bitwise operators are mostly use in -
 - ✓ ○ Embedded systems
 - CPU registers
 - Socket Programming - communication over ports or socket
 - Encryption
 - Compression
 - Graphics programming

Bitwise AND(&) Operator

- The output of bitwise AND is 1 if the corresponding bits of two operands is 1 otherwise the output is 0

AND

8 4 2 1
1 0 1 0
0 1 0 1

A	B	A & B
0	0	0
1	0	0
0	1	0
1	1	1

int a = 10, b = 5.

a b
Result = 0 bitwise AND

① a = 10 = $\begin{matrix} a_3 & a_2 & a_1 & a_0 \\ 1 & 0 & 1 & 0 \end{matrix}$
b = 5 = $\begin{matrix} b_3 & b_2 & b_1 & b_0 \\ 0 & 1 & 0 & 1 \end{matrix}$

$\begin{array}{r} 1010 \\ 0101 \\ \hline 0000 \end{array}$

$(0001)_2 \rightarrow (1)_{10}$

✓

Bitwise AND(&) Operator

Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int a = 2, b = 3, result;

    result = a & b;
    printf("Result: %d", result);

    getch();
    return 0;
}
```

Operation perform on bit level

a (2) = 0000 0000 0000 0000 0000 0000 0000 0010

b (3) = 0000 0000 0000 0000 0000 0000 0000 0011

a & b = 0000 0000 0000 0000 0000 0000 0000 0010 = 2

Output:

Result = 2

Bitwise OR(|) Operator

- The output of bitwise OR is 1 if at least one corresponding bit of two operands is 1 otherwise the output is 0

A	B	A B
✓ 0	0 ✓	0 ✓
✓ 1	0	1 ✓
0	✓ 1	1 ✓
1	✓ 1	1 ✓

$$\begin{array}{r} 1015 \\ 10 = 1010 \\ 5 = 0101 \\ \hline 1 = 1111 \\ \begin{array}{r} 8 \quad 4 \quad 2 \quad 1 \\ 1 \quad 1 \quad 1 \quad 1 \\ \hline = 15_{10} \end{array} \end{array}$$

Bitwise OR(|) Operator

Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int a = 2, b = 3, result;

    result = a | b;
    printf("Result: %d", result);

    getch();
    return 0;
}
```

Operation perform on bit level

a (2) = 0000 0000 0000 0000 0000 0000 0000 0010

b (3) = 0000 0000 0000 0000 0000 0000 0000 0011

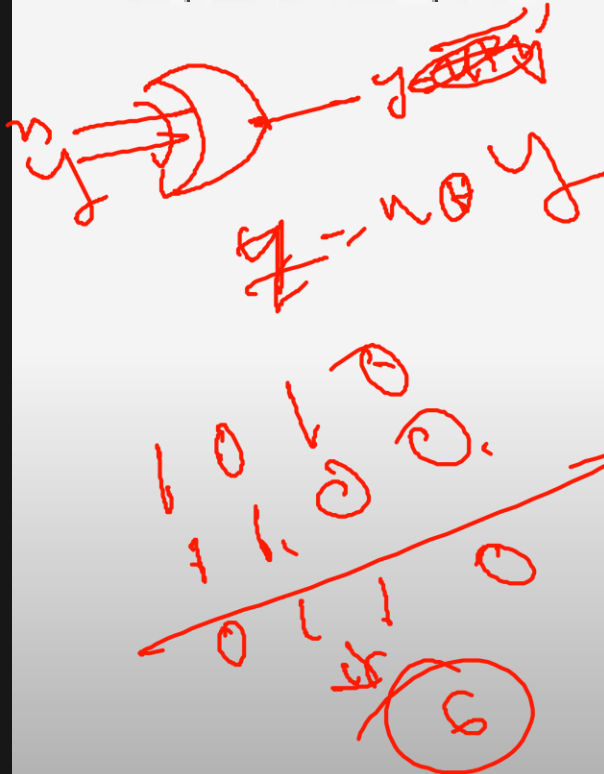
a | b = 0000 0000 0000 0000 0000 0000 0000 0011 = 3

Output:

Result = 3

Bitwise XOR(^) Operator

- The output of bitwise OR is 1 if only one of the corresponding bits is one, otherwise the output is always 0.



A	B	A ^ B
0	0	0
1	0	1
0	1	1
1	1	0

$$\begin{array}{r} a \wedge b \\ 10 = 1010 \\ 5 = 0101 \\ \hline 1111 \\ = 15 \end{array}$$

Bitwise OR(|) Operator

XOR

Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int a = 2, b = 3, result;

    result = a ^ b;
    printf("Result: %d", result);

    getch();
    return 0;
}
```

Operation perform on bit level

a (2) = 0000 0000 0000 0000 0000 0000 0000 0010

b (3) = 0000 0000 0000 0000 0000 0000 0000 0011

a ^ b = 0000 0000 0000 0000 0000 0000 0000 0001 = 1



Output:

Result = 1

Bitwise NOT(~) Operator

- Bitwise NOT(~) operator also called one's one's complement operator
- It is an Unary Operator
- It changes 1 to 0 and 0 to 1

A	~A
0	1
1	0

Bitwise NOT(~) Operator

Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int a = 2, b = 3;

    printf("~a: %d\n", ~a);
    printf("~b: %d\n", ~b);

    getch();
    return 0;
}
```

MSB = 0
+ve
MSB = 1
-ve

8 4 2 1
0 0 1 0

2 = 0011
~2 = 1100 = -

0100
-4

~a = 0010 (convert to decimal)
MSB → 1101
12 = 0010
8 = 1101
3

a (2) = 0000 0000 0000 0000 0000 0000 0000 0010

~a = 1111 1111 1111 1111 1111 1111 1111 1101

Convert to Decimal using 2's Complement

0000 0000 0000 0000 0000 0000 0000 0010
+1

0000 0000 0000 0000 0000 0000 0000 0011

Output:

Result = -3

Left Shift (<<) Bitwise Operator



- Left Shift Operator is used to **shift the bits to the left** of the **First Operand** by the **number of positions specified** in the **Second Operand**
- The bit positions that have been **vacated** by the left shift operation are **filled by zero**
- The bits that are shifted off the end are **discarded**, including the sign bit

First Operand

bits shifted to left

<<

Second Operand

number of position shifted

Left Shift (<<) Bitwise Operator

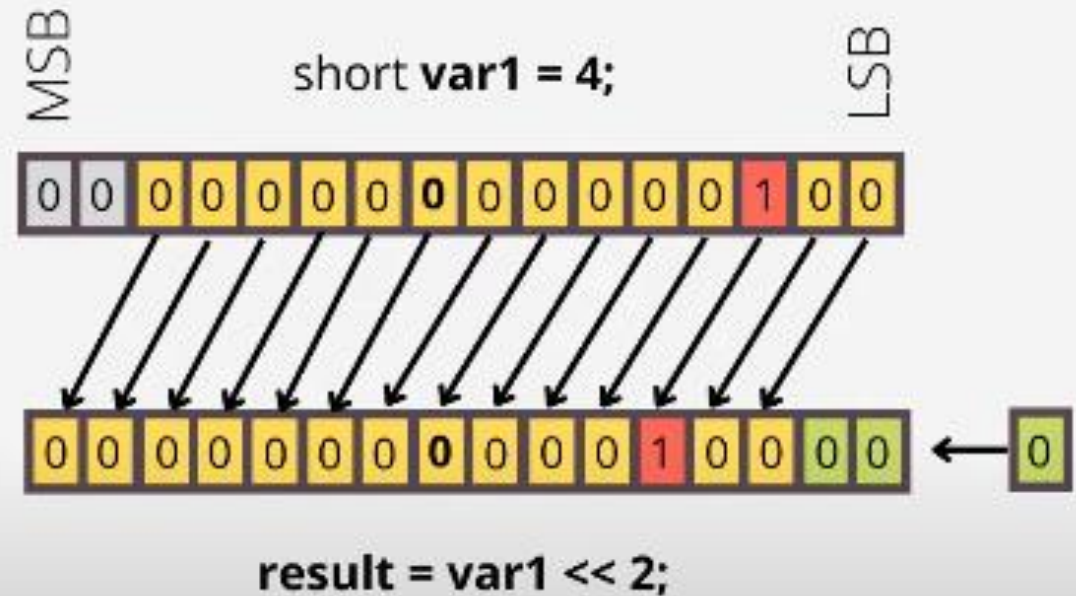
Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    short var1 = 4, result;

    result = var1 << 2;
    printf("result: %d\n", result);

    getch();
    return 0;
}
```



Output:
result = 16

int $x = -10$

left shift operators

$x \ll 1$

(positive number only)

$x = (10)_{10} \rightarrow (C)_{16}$

x discarded

00001010

(convert into 8 bit number)

$\ll 1 =$ 00010100

6 binary to decimal

00010100
 $2^7 \ 2^6 \ 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0$

$6 + 4 = 10$

MCQ

Formula

$a \times 2^b$

when $b = 1$

$$10 \times 2^1 = 10 \times 2 = 20$$

$b = 2$

$$10 \times 2^2 = 10 \times 4 = 40$$

$b = 5$

$$10 \times 2^5 = 10 \times 32 = 320$$

$x \ll 2$
two bit shift

x

00001010

2 bit = 00101000

$2^7 \ 2^6 \ 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0$

$$1 \times 2^5 + 1 \times 2^3$$

$$= 32 + 8$$

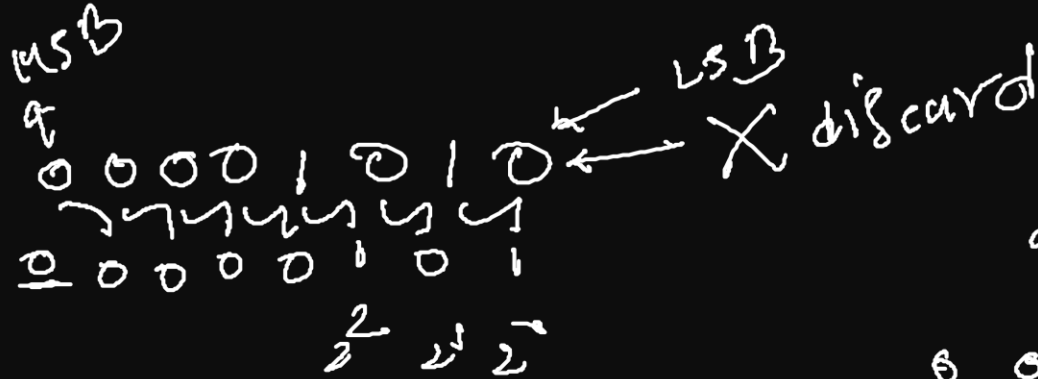
$$= 40$$

Logical Right Shift

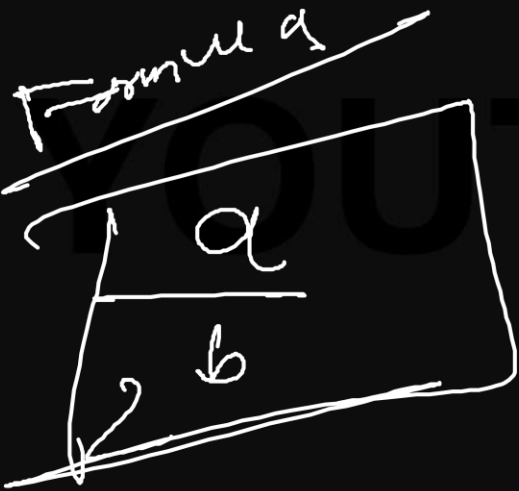
int a = 10

a >> 1 (MSB start)

10 =



a >> 2



$$1 \times 2^2 + 1 \times 2^1$$

$$4 + 2 = 6$$

b = 6

$$\frac{10}{2^1} = \frac{10}{2} = 5$$

2

division always
 returns integer
 if both operand
 are int

b = 2

$$\frac{10}{2^2} = \frac{10}{4} = 2$$

10 / 4 = 2 R 2

-27 >> 2

-27 = 27 = 11011

Right shift

$$\begin{array}{r} 2 \overline{) 27} \\ \underline{13} \\ 2 \overline{) 13} \\ \underline{6} \\ 2 \overline{) 6} \\ \underline{3} \\ 2 \overline{) 3} \\ \underline{1} \\ 2 \overline{) 1} \\ \underline{0} \end{array}$$

Short cut

$$\begin{array}{r} 16 \ 8 \ 4 \ 2 \ 1 \\ \hline 1 \ 1 \ 0 \ 1 \ 1 \end{array}$$

Memory Store

1

= 00011011

1's = 11100100

2's = +1

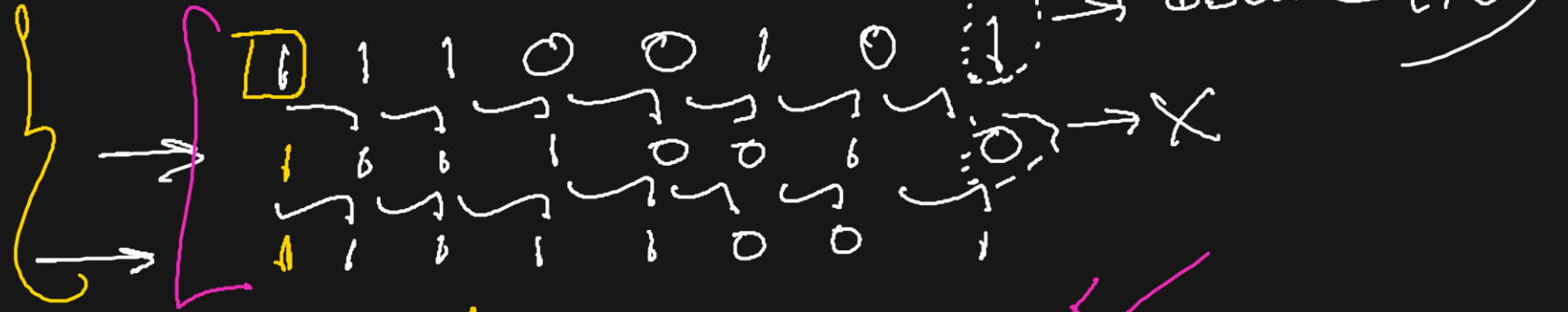
1100101

Pre-Saved bit (fixed)

MSB = 1 indicate result

negative number

2 shifting



3

1's = 00000110

2's = 0000111

-7

negative (Left Shift)

$$-27 \ll 1$$

$$-27 = 00011011$$

$$\oplus = 11100100$$

$$+ 1$$

$$11100101$$

fixed

$$11100101$$

$$11100101$$

$$11100101$$

$$+ 1$$

$$00110110$$

$$\begin{array}{r} 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \\ 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \end{array}$$

$$\begin{array}{r} 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \end{array}$$

$$-54$$

Right Shift (<<) Bitwise Operator

- Right Shift Operator is used to shift the bits to the right of the First Operand by the number of positions specified in the Second Operand
- The bit positions that have been vacated by the right shift operation are filled by zero
- The bits that are shifted off the end are discarded



First Operand

>>

Second Operand

Right Shift (>>) Bitwise Operator

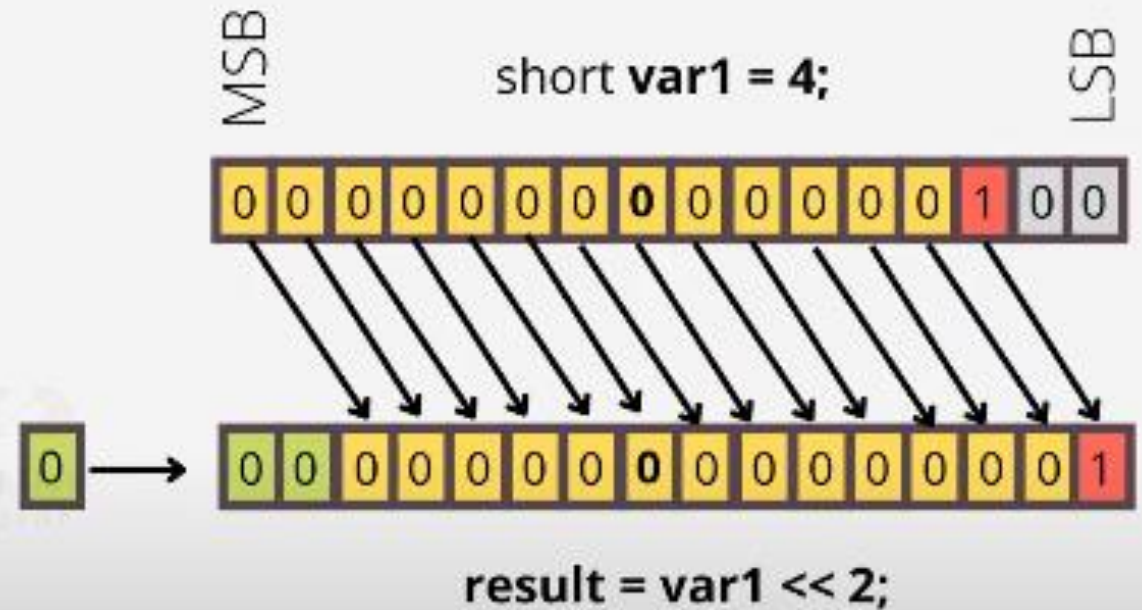
Example

```
#include <stdio.h>
#include <conio.h>

int main()
{
    short var1 = 4, result;

    result = var1 >> 2;
    printf("result: %d\n", result);

    getch();
    return 0;
}
```



Output:
result = 1

COMMA OPERATOR

Special Operators

Special operators	<code>?:</code> <code>&</code> <code>*</code> <code>sizeof()</code> <code>,</code>
-------------------	--



Tutorials
Point

Comma (,) Operator

- Comma can be used as
 - a Separator
 - an Operator
- The comma operator has **left-to-right** associativity and has the **lowest** precedence
- Comma is a Binary Operator, **evaluate** its **first operand** and **reject the result** then **evaluate the second Operand** and **return the value**

